Min Max Stack Construction

AlgoExpert · Medium · Stacks

The problem

- A stack with O(1) push, pop, peek, getMin and getMax by storing the running min/max alongside each element.

Intuition

- Store the current min and max alongside each pushed value so they're always available.

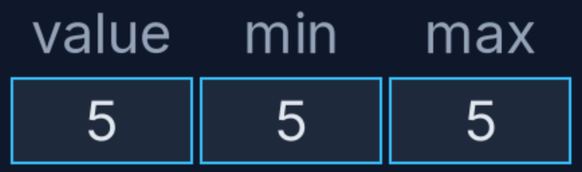
Brute force vs optimal

- Scan the stack for min/max → O(n)
- Store (value, min, max) per entry → O(1) all ops
- Extra O(n) space

Approach

- 1. Push value with updated min and max
- 2. Pop removes both
- 3. Peek, getMin and getMax read the top

Worked example




```
push 5
min = 5, max = 5
```

push 5 → min 5, max 5

Worked example

value	min	max
7	5	7
5	5	5

```
push 7
min = 5, max = 7
```

push 7 → min 5, max 7

Worked example

value	min	max
2	2	7
7	5	7
5	5	5

```
push 2
min = 2, max = 7
```

push 2 → min 2, max 7

Worked example

value	min	max
7	5	7
5	5	5

```
pop
min = 5, max = 7
```

pop → min 5, max 7

Worked example

value	min	max
9	5	9
7	5	7
5	5	5

```
push 9
min = 5, max = 9
```

push 9 → min 5, max 9

Java solution

```java
public class MinMaxStackConstruction {
    public static class MinMaxStack {
        private final List<int[]> st = new ArrayList<>();

        public int peek() { return st.get(st.size() - 1)[0]; }
        public int pop() { return st.remove(st.size() - 1)[0]; }
        public void push(int v) {
            if (st.isEmpty()) st.add(new int[]{v, v, v});
            else { int[] t = st.get(st.size() - 1); st.add(new int[]{v, Math.min(v, t[1]), Math.max(v, t[2])}); }
        }
        public int getMin() { return st.get(st.size() - 1)[1]; }
        public int getMax() { return st.get(st.size() - 1)[2]; }
    }
}
```

Complexity

- Time: O(1) per op
- Space: O(n)

Edge cases & pitfalls

- Empty stack behaviour
- Pop restores previous min/max automatically

Interview tips & follow-ups

- Follow-up: min-stack with O(1) extra space (encoding trick)

Recap

- Store the current min and max alongside each pushed value so they're always available.
- Complexity: O(1) per op